

IC526 Advanced Compilers

Lec 04: Automata Basics I

Yoonseung Kim

DGIST EECS

Fall 2026

Announcement

- On Thursday:
 - Each of you will be given a private GitHub repository.
 - HW1 will be posted on the course website.

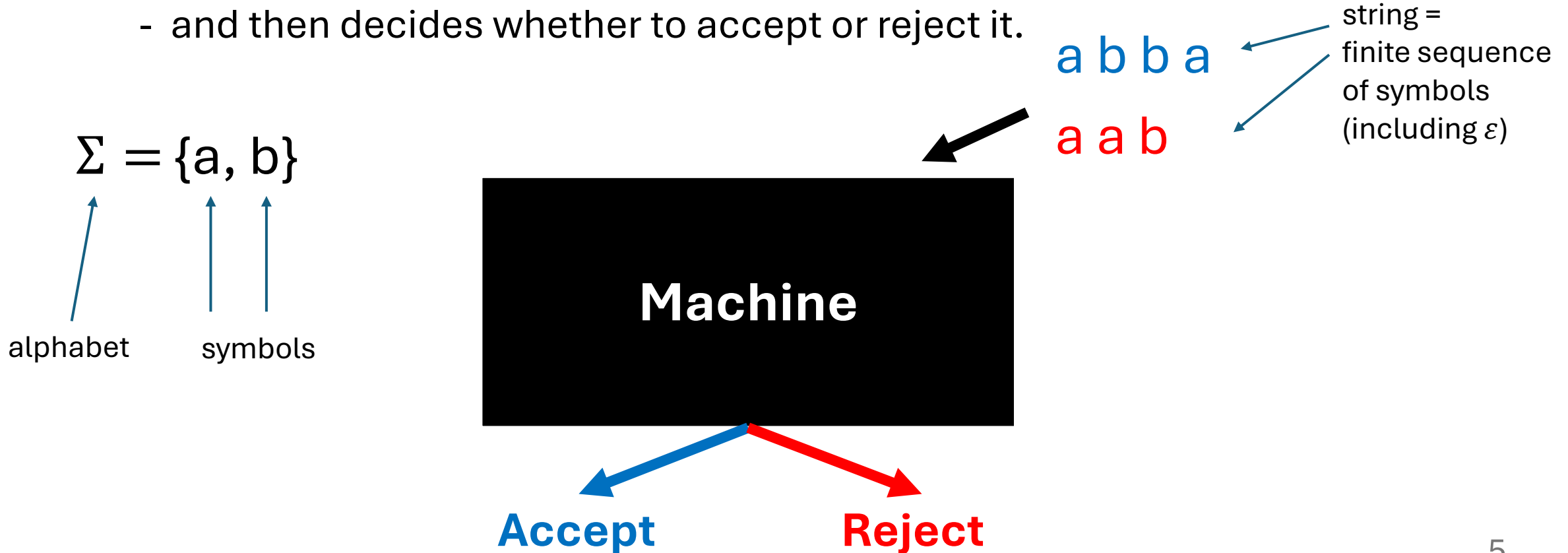
Where We Are

- Lec 01: The course overview
- Lec 02: OCaml – The development language
- Lec 03: Rust – The source language
- Lec 04: Automata basics – Minimal theory for lexer/parser design

Deterministic Finite Automata (DFA)

A Machine That Accepts Some Strings

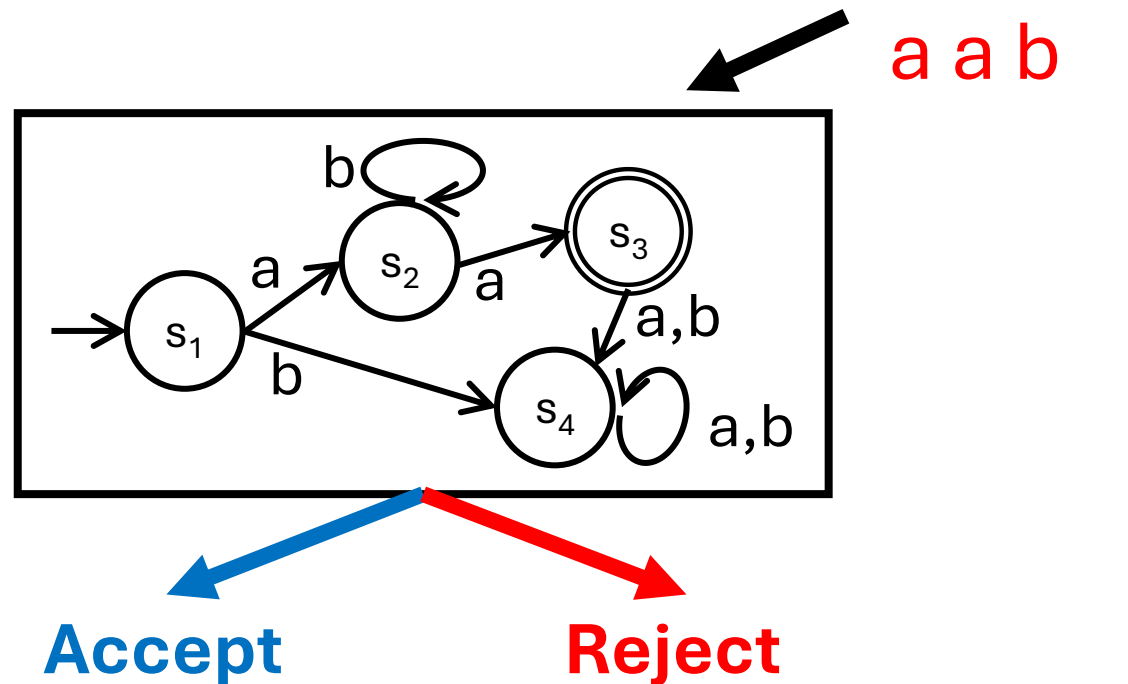
- Think about a machine that
 - takes a string as a *stream* of symbols,
 - and then decides whether to accept or reject it.



A Machine That Accepts Some Strings

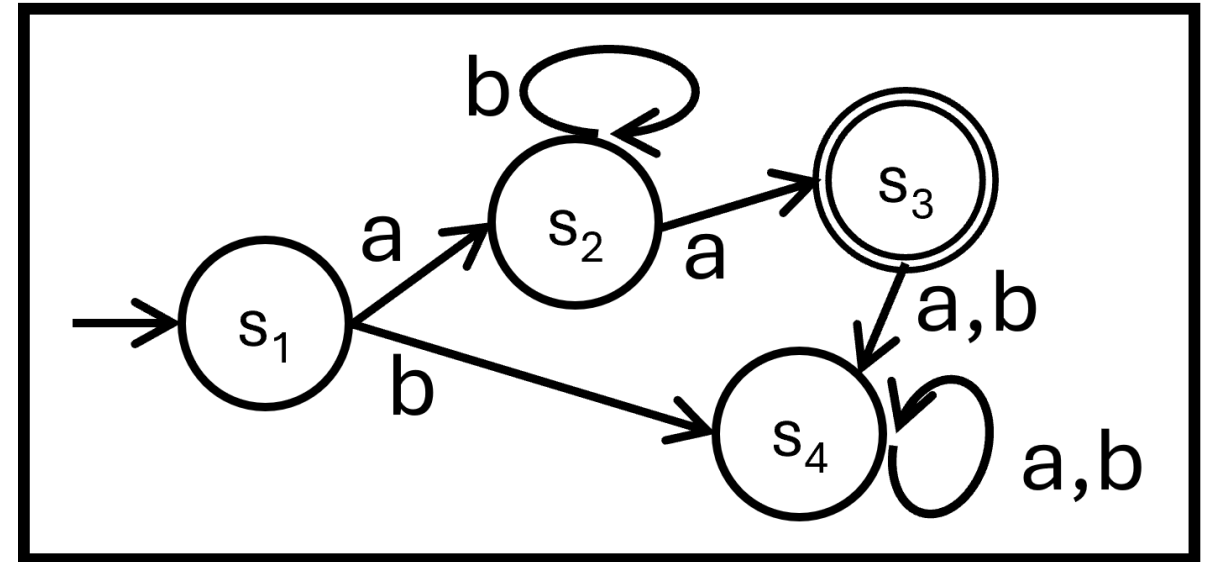
- Think about a machine that
 - takes a string as a *stream* of symbols,
 - and then decides whether to accept or reject it.

$\Sigma = \{a, b\}$



DFA: Formal Definition

- A DFA $M = (Q, \Sigma, \delta, q_0, F)$
 - Q (Finite set of states)
 - Σ (Finite set of symbols)
 - $\delta: Q \times \Sigma \rightarrow Q$
(Transition function)
 - $q_0 \in Q$ (Initial state)
 - $F \subseteq Q$ (Final states)

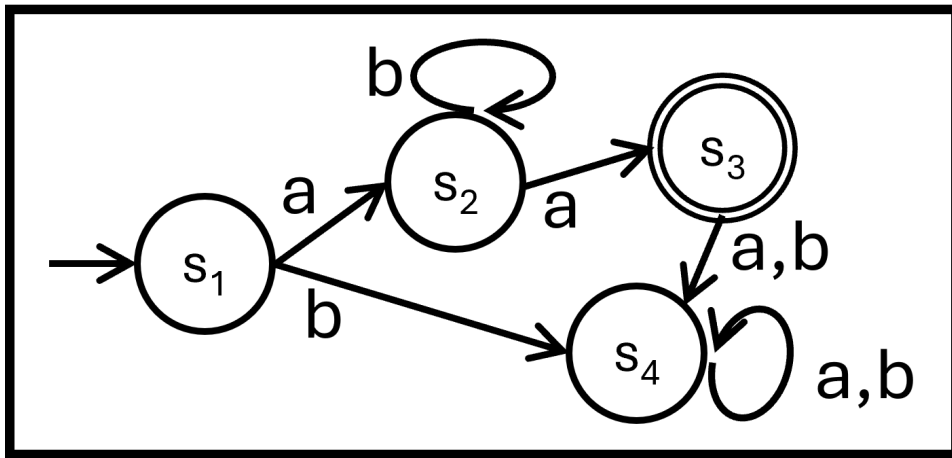


Exercise:

Let this automaton be $M = (Q, \Sigma, \delta, q_0, F)$.
Identify each component.

DFA Exercises

- Which set of strings does this automaton accept?



We say “M **recognizes** the **language** { aa, aba, ... }”

$$L(M) = \{ aa, aba, \dots \} \subseteq \Sigma^*$$

- Build an automaton that recognizes each of the languages given below.

$$L1 = \{ aab, aabab, aababab, \dots \}$$

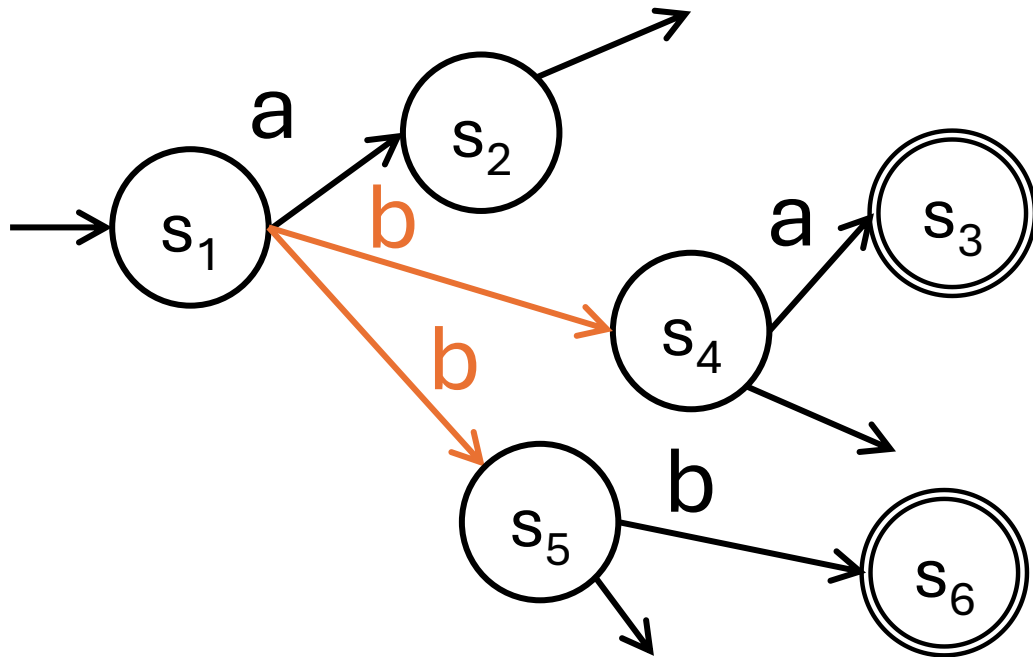
$$L2 = \{ aa, abba \}$$

$$L3 = L1 \cup L2$$

Nondeterministic Finite Automata (NFA)

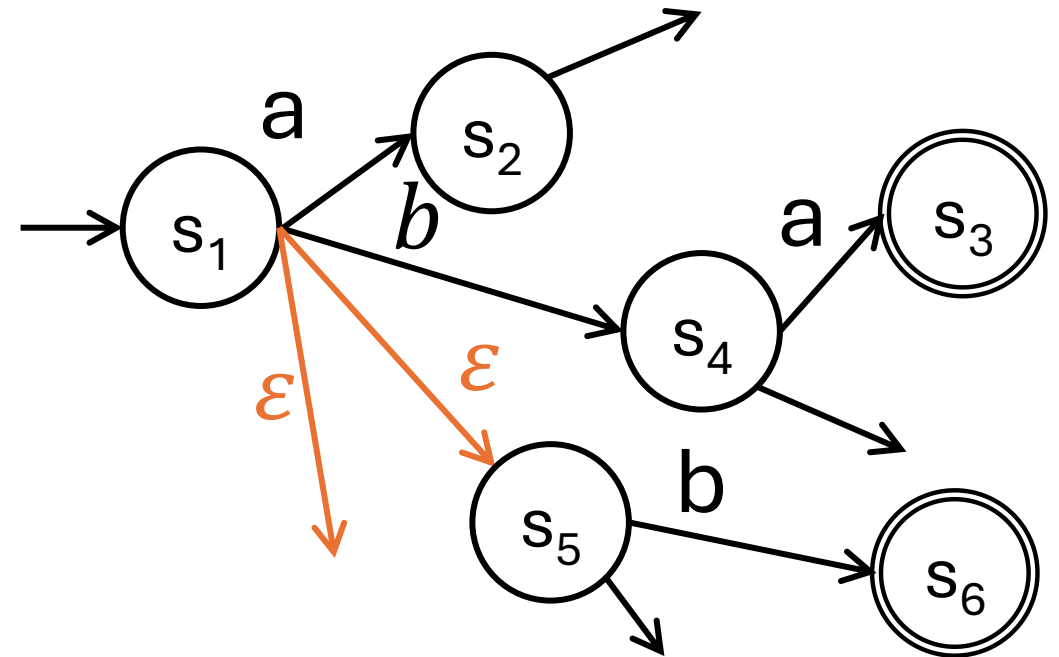
Nondeterministic Finite Automata with ϵ (NFA- ϵ)

- Nondeterministic transitions



$L(M)$ includes **both** “ba” **and** “bb”
- If there **exists at least one execution** that accepts s, then M accepts s.

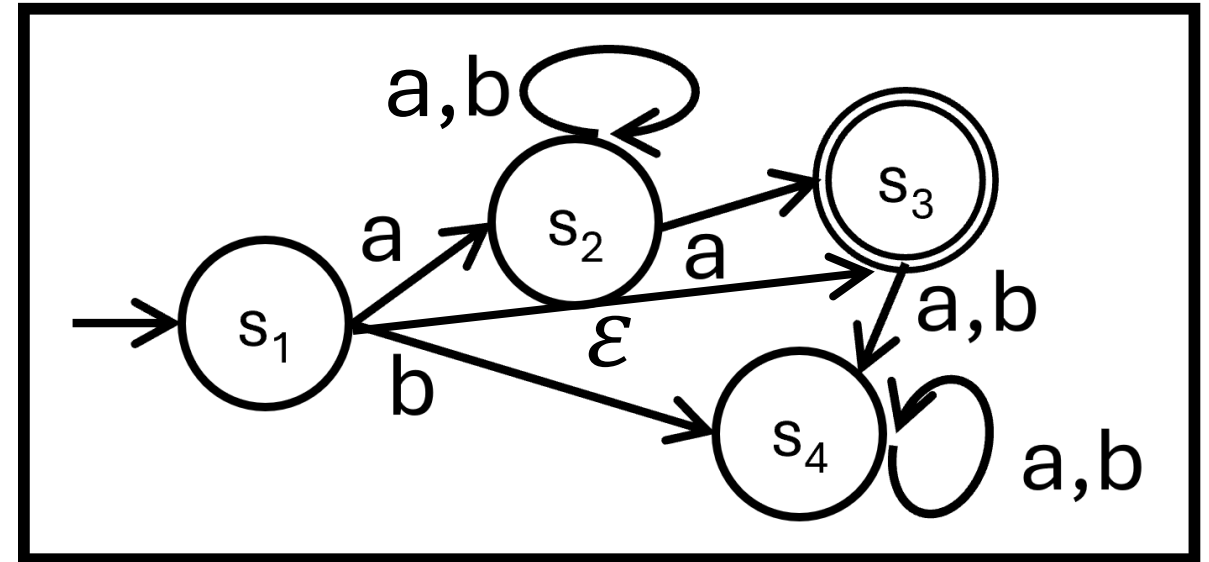
- ϵ -transitions



$L(M)$ includes “ba” **and** “b”
- M can move from s_1 to s_5 **without consuming a symbol.**

NFA- ϵ : Formal Definition

- An NFA $M = (Q, \Sigma, \delta, q_0, F)$
 - Q (Finite set of states)
 - Σ (Finite set of symbols)
 - $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$
(Transition function) empty?
 - $q_0 \in Q$ (Initial state)
 - $F \subseteq Q$ (Final states)



Exercise:

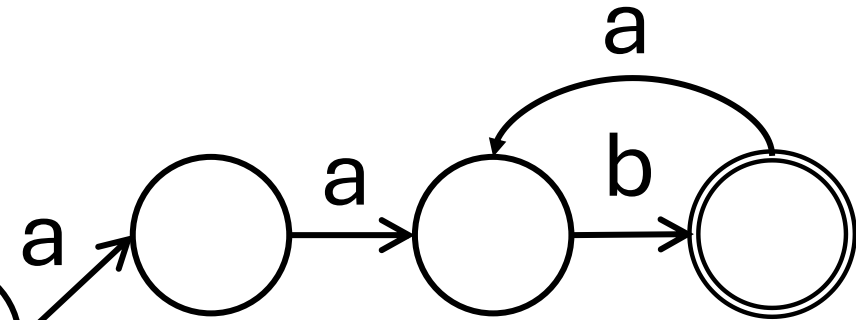
Let this automaton be $M = (Q, \Sigma, \delta, q_0, F)$.
Identify each component.

Note that NFA is a generalization of DFA
> **A DFA machine** is also an **NFA machine**.

NFA- ϵ Machine Recognizing $L1 \cup L2$

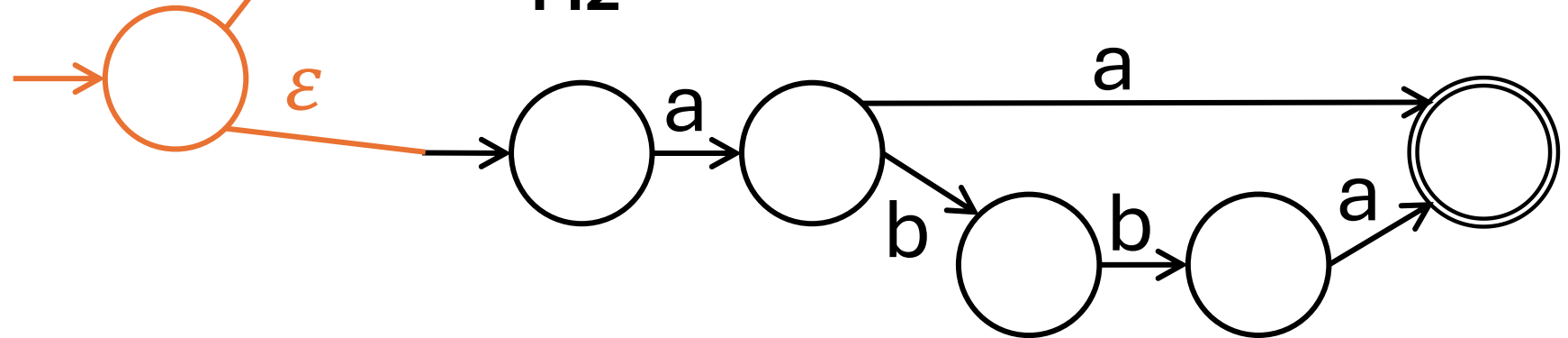
- $L1 = \{ aab, aabab, aababab, \dots \}$

M1



- $L2 = \{ aa, abba \}$

M2



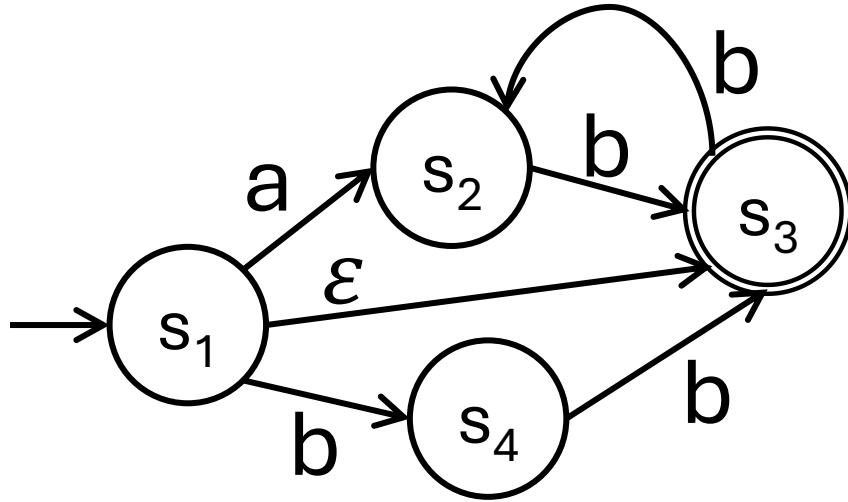
Given M1 & M2, we can easily construct M such that:

$$L(M) = L(M1) \cup L(M2)$$

Surprising Fact 1

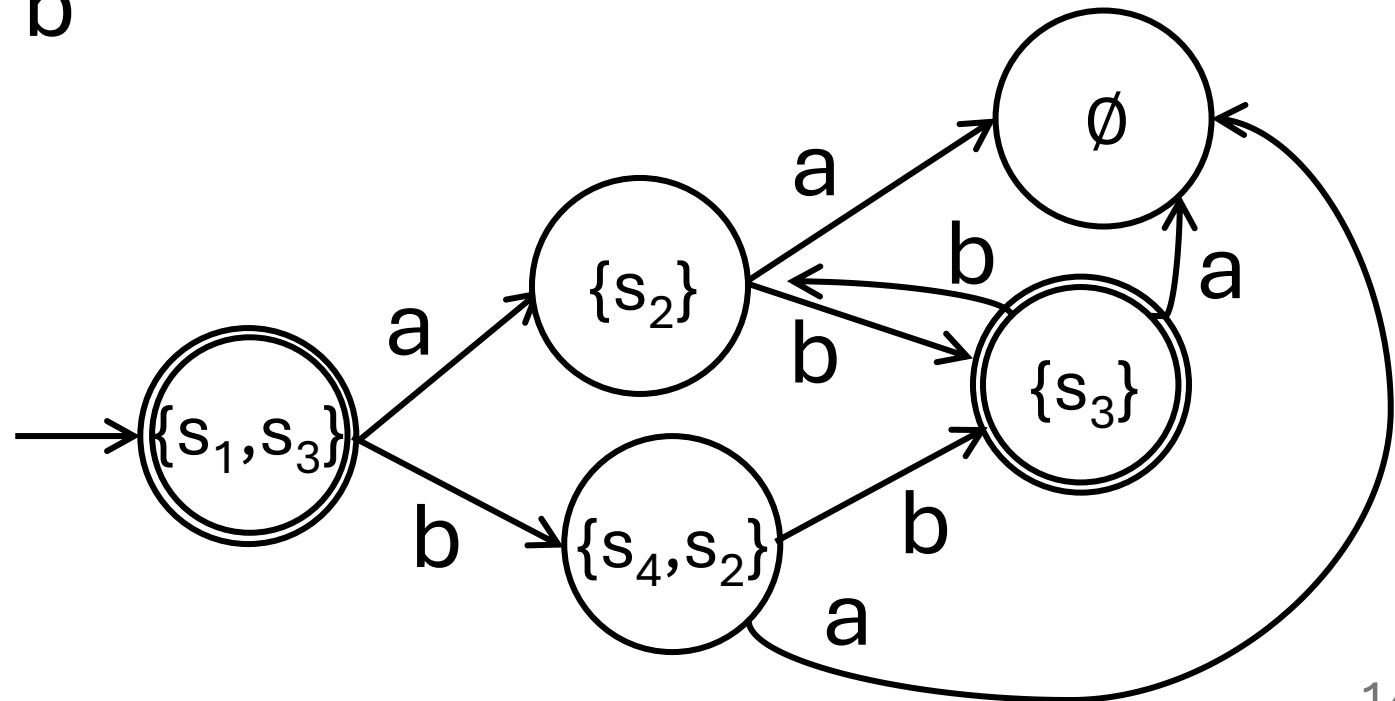
- Given DFA, there exists NFA that recognizes the same language.
 - Trivial, since every DFA machine is an NFA machine.
- NFA $\rightarrow$ exists DFA
 - Surprising!
- The equivalence theorem: expressivity of DFA = that of NFA

Why? DFA Construction Exists



Each DFA state represents the set of **reachable states** for the given NFA with the identical string.

This is possible due to the **finiteness** of the NFA states.



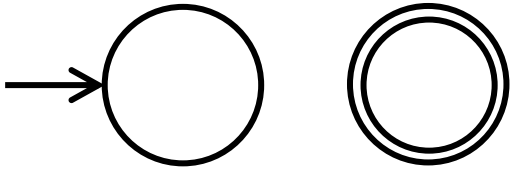
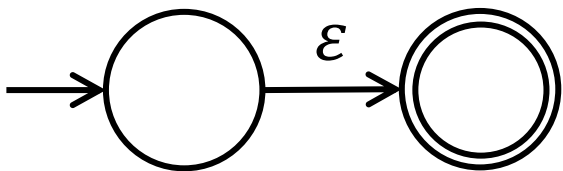
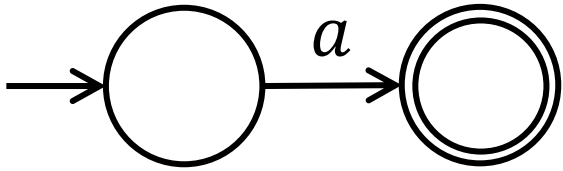
Regular Expressions

Describing A Set of Strings

- It's hard to represent a language as a bare set of strings.
- Regular expressions (RE) provide an effective representation of languages.

Constructors of Regular Expression

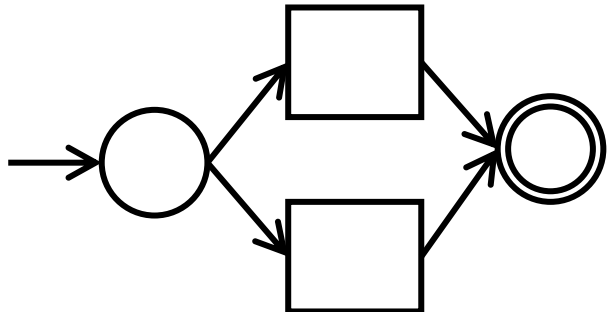
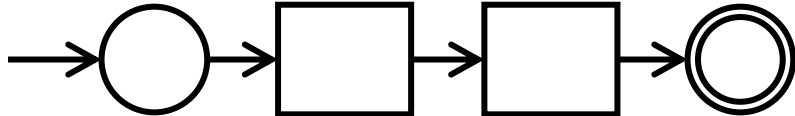
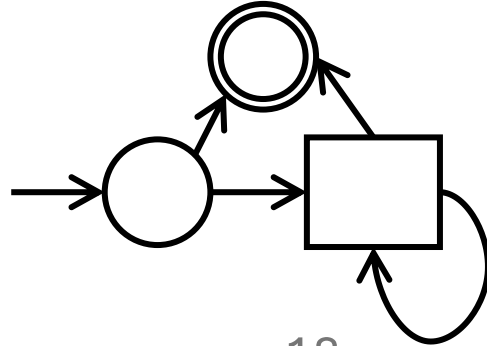
- Base cases

Regular Expression	Empty set $\emptyset$	Empty string ε	Single symbol $a (\in \Sigma)$
Corresponding Language	$L(\emptyset) = \emptyset$	$L(\varepsilon) = \{\varepsilon\}$	$L(a) = \{a\}$
An NFA that recognizes L			

Single final state

Constructors of Regular Expression

- Inductive cases

Regular Expression	Union $R_1 R_2$	Concatenation $R_1 \cdot R_2$	Kleene Star (Repetition) R^*
Corresponding Language	$L(R_1 R_2) = L(R_1) \cup L(R_2)$	$L(R_1 \cdot R_2) = \{st \mid s \in L(R_1), t \in L(R_2)\}$	$L(R^*) = \bigcup_{n \geq 0} L^n(R)$
An NFA that recognizes L			

Regular Expression Examples

$ab \mid xy$

$\{ 'ab', 'xy' \}$ ϵ : empty string

$(1a)^*$

Interpretation

$\{ \epsilon, '1a', '1a1a', '1a1a1a', \dots \}$

$1a(1a)^*$



$\{ '1a', '1a1a', '1a1a1a', \dots \}$

$(1a \mid 2b)^*$

$\{ \epsilon, '1a', '2b', '1a1a', '1a2b', '2b1a', \dots \}$

$\text{DIGIT} = 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\{ '0', '1', '2', '3', '4', '5', '6', '7', '8', '9' \}$

DIGIT^*

$\{ \epsilon, '0', '1', \dots, '9', '00', '01', \dots, '123', \dots \}$

Surprising Fact 2

- **Theorem (Kleene).** For any language $L \subseteq \Sigma^*$, the following are equivalent:
 - (1) $L = L(r)$ for some regular expression r
 - (2) $L = L(N)$ for some NFA- ϵ N
 - (3) $L = L(M)$ for some DFA M
 - Such a language is called **regular**.
- > We have dealt with (2) $\Rightarrow$ (3), (3) $\Rightarrow$ (2), (1) $\Rightarrow$ (2).
> (2) $\Rightarrow$ (1) will complete the proof
(It's true but will not be covered in this lecture.)

The **KEY** Takeaway

- If you can write your language as a **regular expression**, you can **automatically construct a DFA** that recognizes it.

Alpha = 'A' | 'B' | ... | 'Z' | 'a' | 'b' | ... | 'z'

Digit = '0' | '1' | '2' | ... | '9'

Ident = (Alpha | '_')(Alpha | Digit | '_')*

